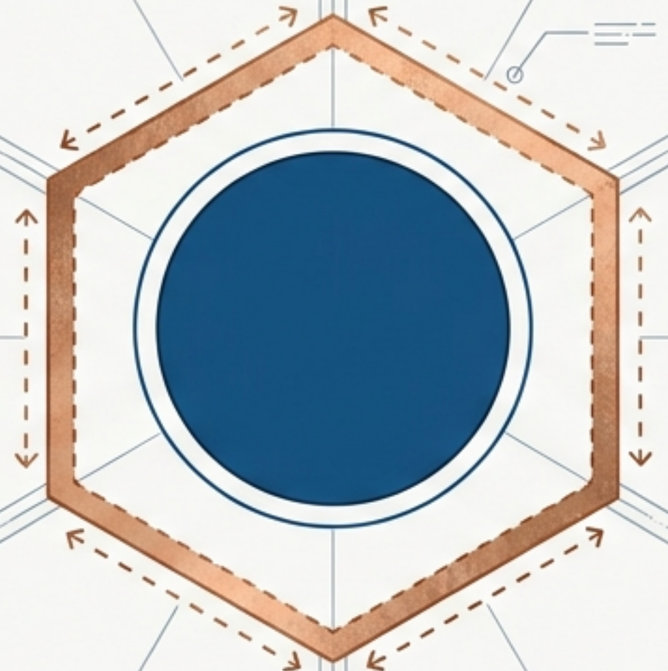
The Blueprint and The Boundary

Organizing Systems to Survive the Cost of Change.



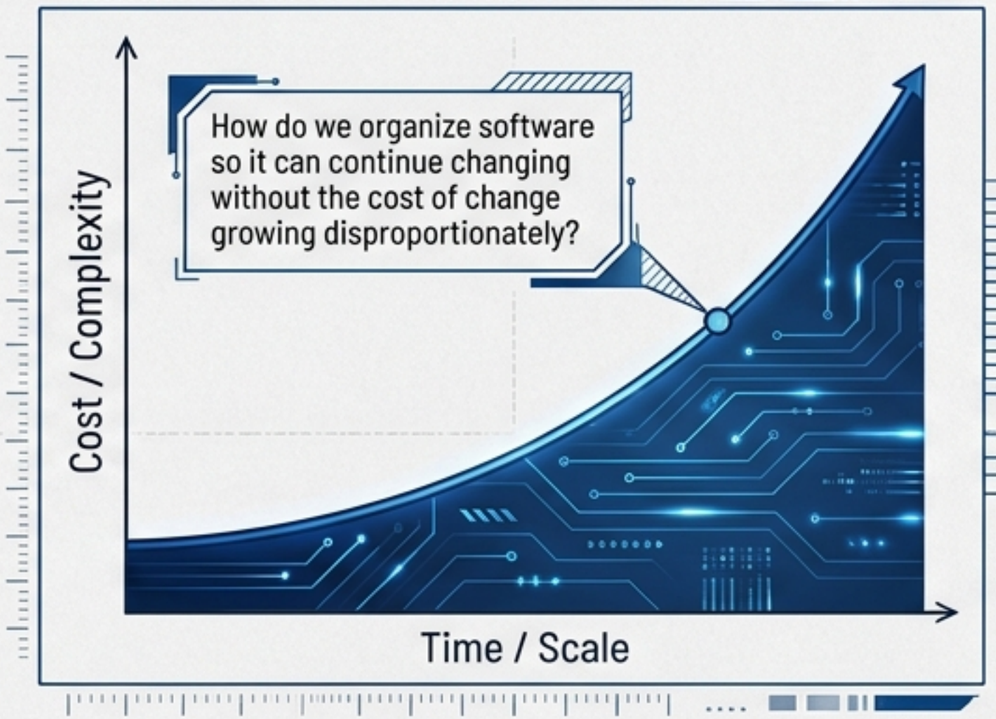
From Code to System Structure.

Gemini Notebook

These are Solutions, Not Problems.

Architecture shouldn't start with a technology choice.

The Fundamental Question.

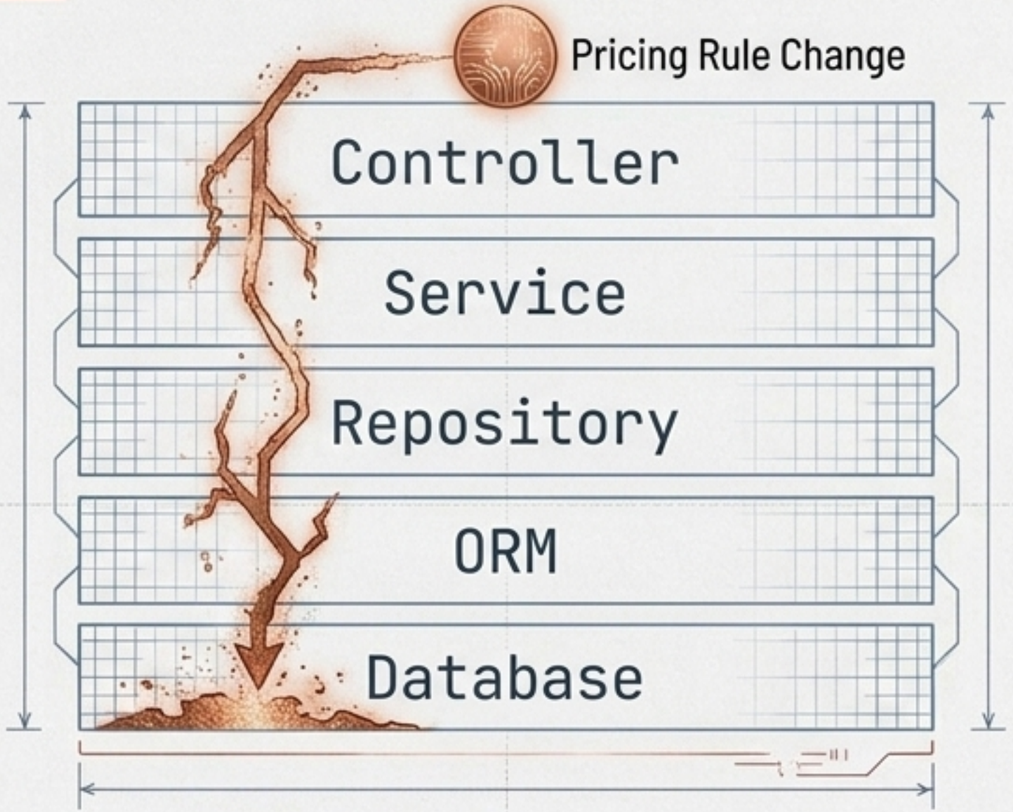


Key Insight

Good architecture doesn't correctly predict the future. It creates optionality—making it cheap to be wrong about the future.

Gemini Notebook

The Anatomy of a Change.



Key Insight

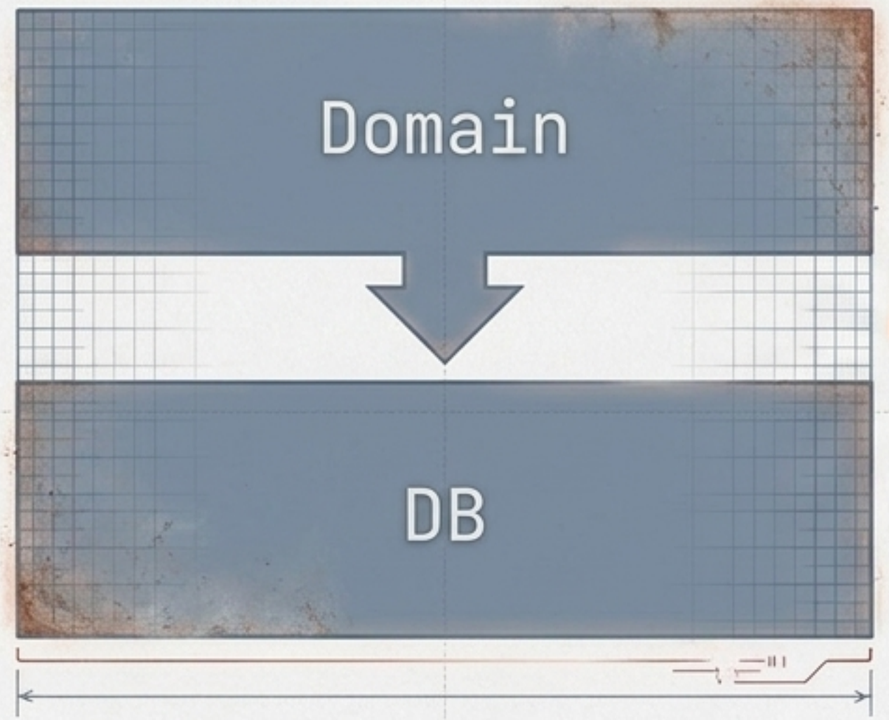
The Layered Illusion. A system can be perfectly layered yet heavily coupled. A simple business change forces modifications across APIs, ORMs, and external services.

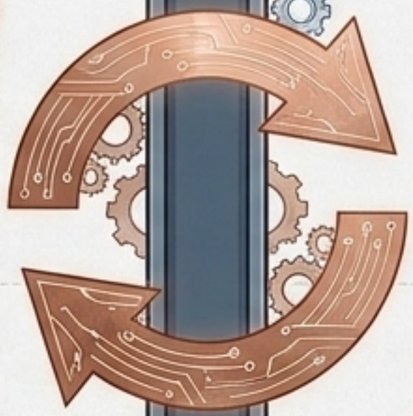
Key Insight

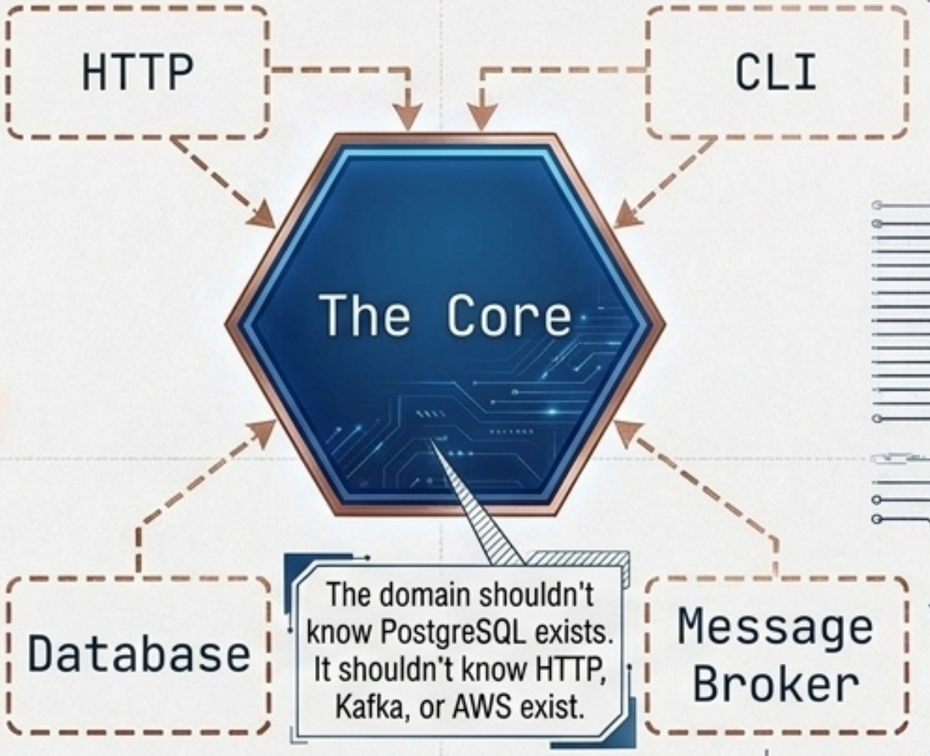
The Healthy Boundary. A healthy architecture ensures that a conceptually small business change crosses as few technical boundaries as possible.

Gemini Notebook

Flipping the Dependencies.







Key Insight

The Dependency Rule: What the system does must be isolated from how it talks to the outside world. Business decisions must not depend on technological decisions.

Gemini Notebook

Functional Core vs. Imperative Shell.

How do we prevent business logic from drowning in state, side effects, and dependencies?

The Functional Core

Receives values, makes decisions, produces values.

No DB access, no API calls, no clock, no external state modification.

f: Input → Output (Deterministic).

The Imperative Shell

Reads requests, loads data, gets current time.

Calls the core, then executes the resulting effects.

Coordinates the outside world.

This is an architectural principle, not language-dependent. Works in Java, C#, Kotlin, Go, TypeScript—not just Haskell.

Gemini Notebook

The Inversion of Action: Decide vs. Execute (Customer Example)

1. Initial Design — Everything Mixed

```
async Task UpdateCustomer()
{
    var existing = await db.GetCustomer();
    if (existing.Name != newCustomer.Name ||
        existing.Email != newCustomer.Email)
    {
        existing.Name = newCustomer.Name;
        existing.Email = newCustomer.Email;
        await db.UpdateCustomer(existing);
    }
    if (existing.Email != newCustomer.Email) {
        await email.Send(newCustomer.Email,
        'Please verify your new email');
    }
}
```

Problems

- UpdateCustomer() doesn't receive inputs explicitly.
- Doesn't return decision result.
- Directly depends on db and email.
- It's async because business logic is mixed with I/O.
- Hard to test rules without database and email server.
- It's unclear what the function decides and what it executes.
- Central Problem: Business rule is hidden within side effects.

Core decides. Shell executes.

2. Core — Turn Decision into Data

```
UpdateCustomerResult UpdateCustomer(Customer existing, Customer updated)
{
    if (existing.Email != updated.Email) {
        return new(UpdateCustomerDecision.UpdateAndSendEmail, updated,
                new EmailMessage(updated.Email, 'Please verify...'));
    }
    if (existing.Name != updated.Name) {
        return new(UpdateCustomerDecision.UpdateOnly, updated, null);
    }
    return new(UpdateCustomerDecision.DoNothing, null, null);
}
```

Customer + Customer → explicit decision → UpdateCustomerResult

3. Shell — Execute the Decision

```
async Task UpdateCustomer(Customer updated)
{
    var existing = await db.GetCustomer(updated.Id);
    var result = UpdateCustomer(existing, updated);
    switch (result.Decision) {
        case DoNothing: break;
        case UpdateOnly:
            await db.UpdateCustomer(result.Customer); break;
        case UpdateAndSendEmail:
            await db.UpdateCustomer(result.Customer);
            await email.Send(result.Message); break;
    }
}
```

IMPERATIVE SHELL

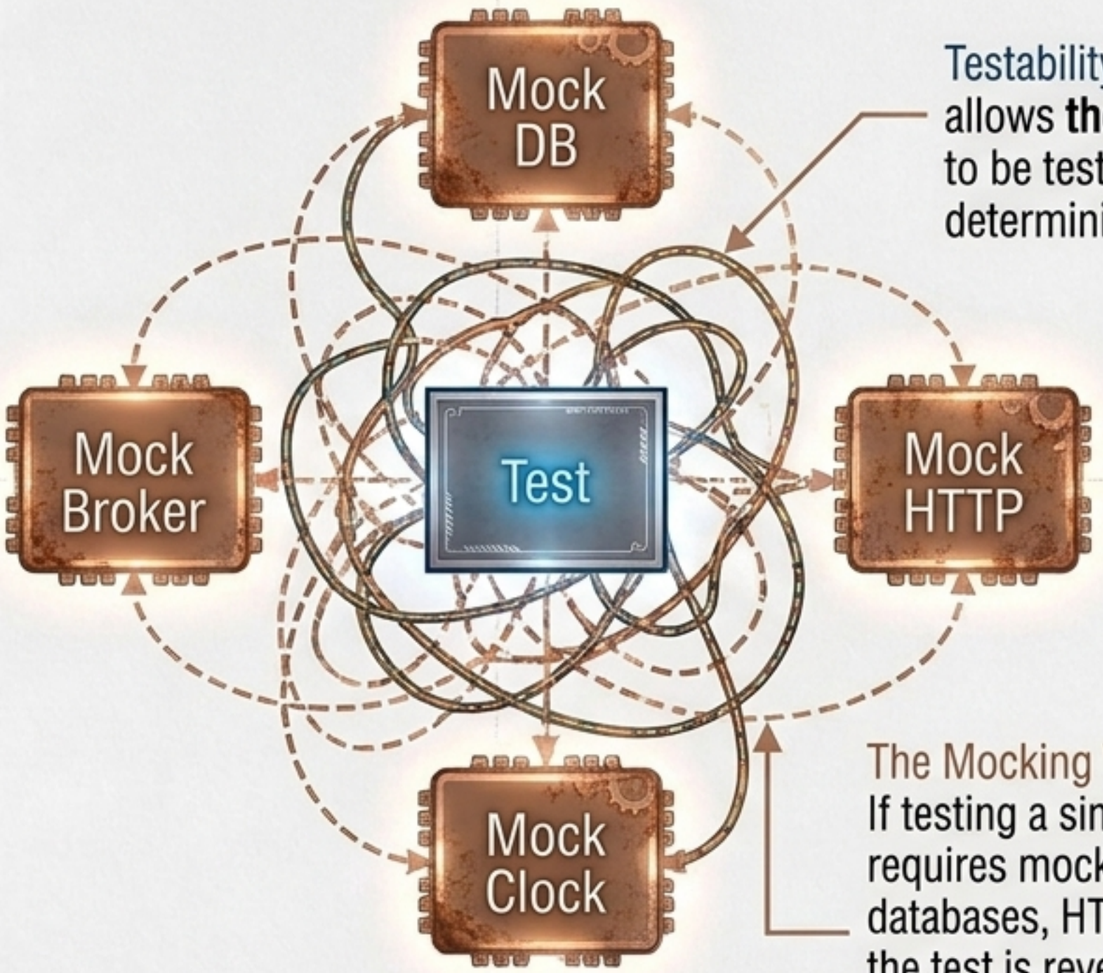
DB → reads Customer → decide CORE → executes decision → DB, Email

I/O → pure decision → I/O → pure decision

The Shell coordinates the workflow; each decision remains a pure and testable function.

Gemini Notebook

TDD as an Architectural Sensor.



Coupled Architecture

Testability via Architecture. The Functional Core allows **the vast** majority of system behavior to be tested without infrastructure. They are deterministic, lightning-fast, and isolated.

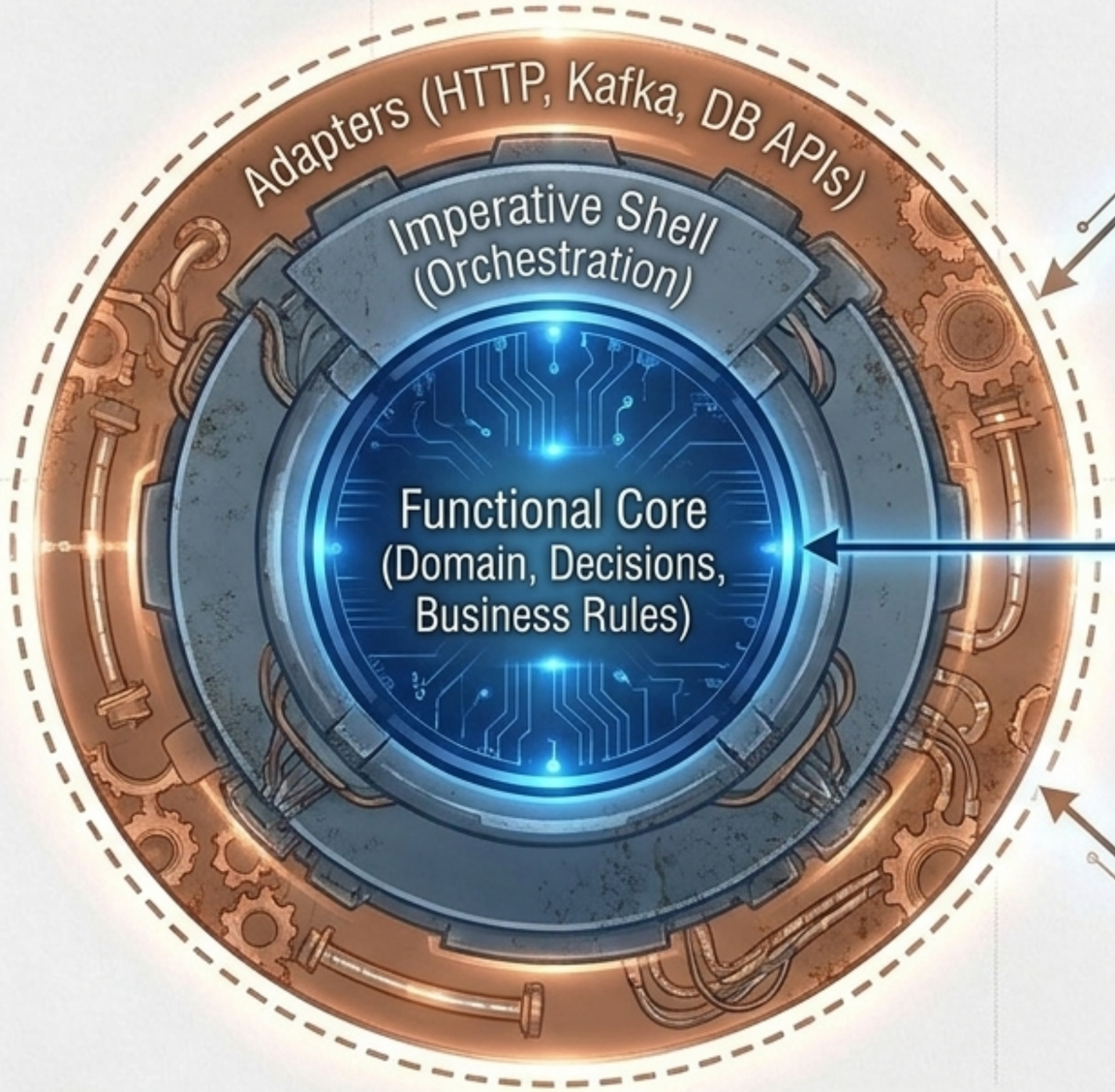
The Mocking Warning.
If testing a simple rule requires mocking databases, HTTP, and clocks, the test is revealing a design flaw: the business decision is too dependent on the external world.

Testability via Architecture.
The Functional Core allows the vast majority of system behavior to be tested without infrastructure. They are deterministic, lightning-fast, and isolated.

Functional Core

Gemini Notebook

The Universal Principle: Push Effects to the Edges.



Ports & Adapters answers:
How do we isolate the core from external mechanisms?

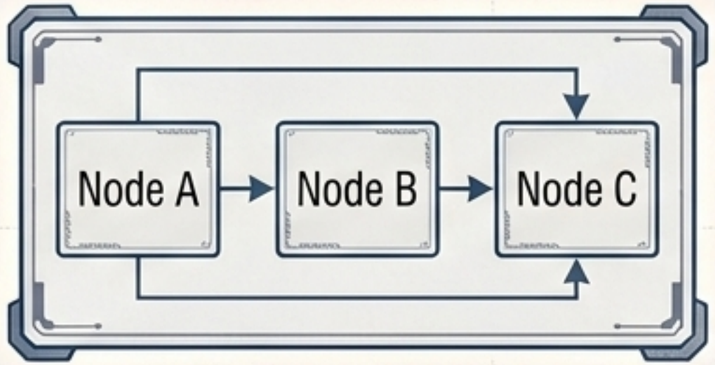
Functional Core answers:
How do we keep the core simple, deterministic, and effect-free?

Clean Architecture answers:
How do we organize dependencies around business policies?

Gemini Notebook

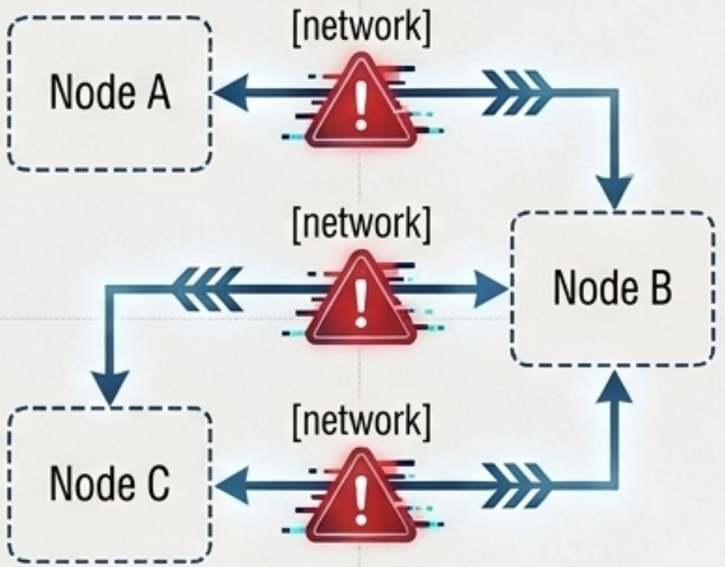
The Distributed Monolith Trap.

Topology A (Monolith)



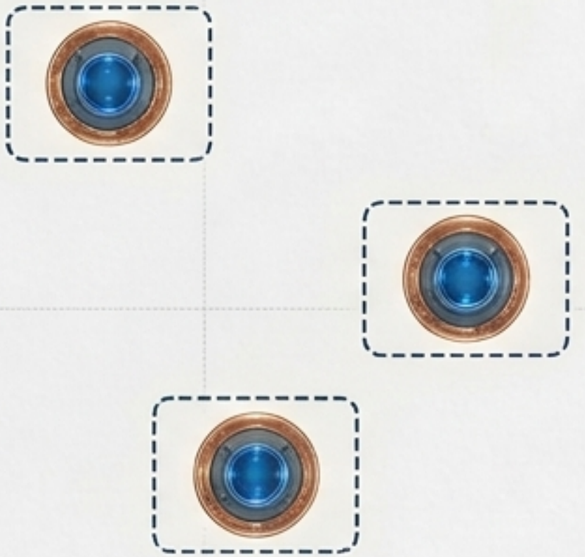
High coupling, but calls are local and cheap.

Topology B (Distributed Monolith)



High coupling, PLUS latency, network failures, distributed debugging, and deployment coordination. You distribute complexity without reducing it.

Topology C (Microservices)

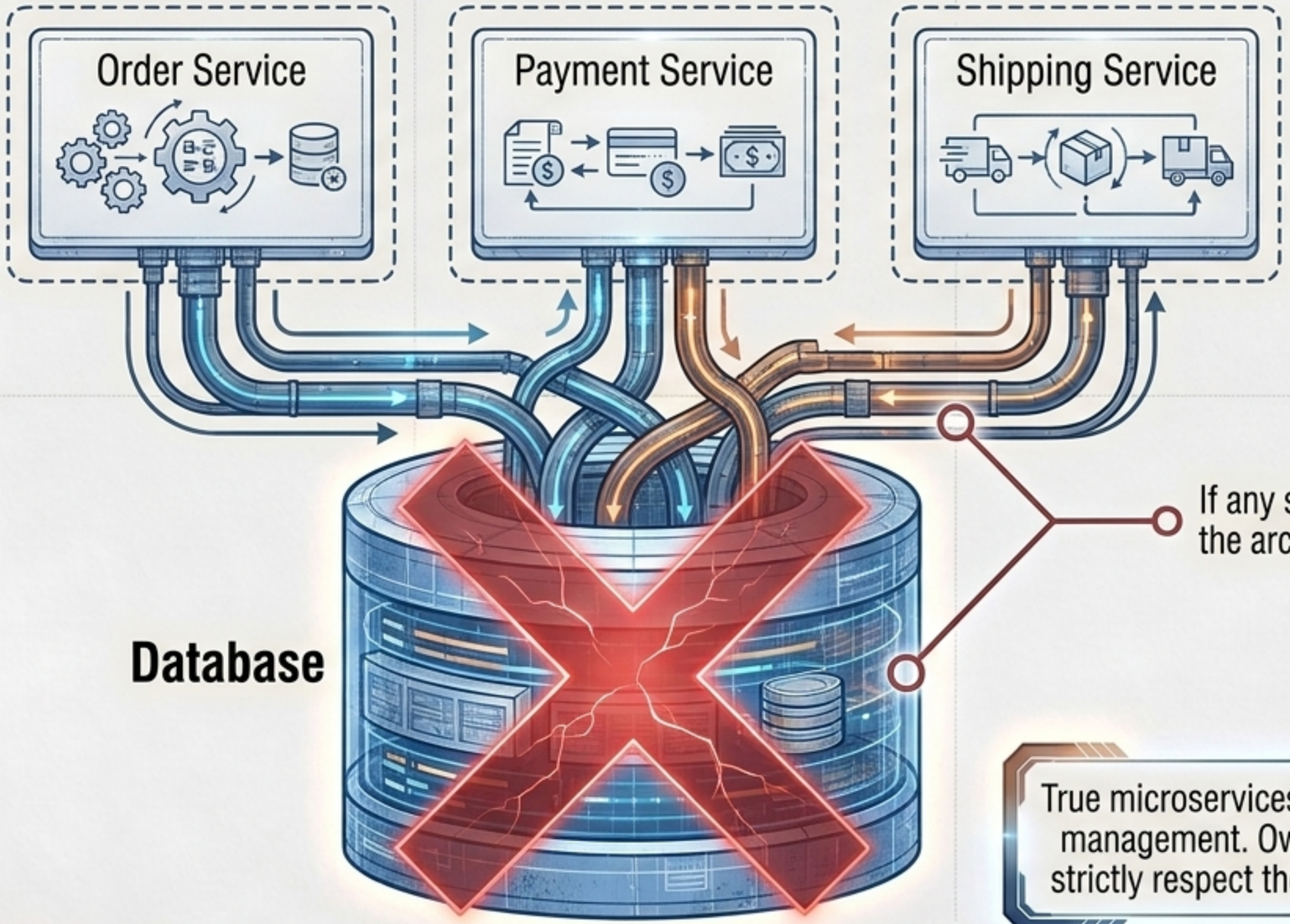


Teams can develop, test, deploy, and operate independently.

> "Microservices carry a premium of complexity. They are not just classes in different processes."
> — Martin Fowler

Gemini Notebook

Shared Databases are Fake Boundaries.



Key Equation Box

Service Boundaries ≠ Data Boundaries

If any service can alter the tables of another, the architectural boundary is an illusion.

True microservices require decentralized data management. Ownership of the data must strictly respect the boundary of the service.

Gemini Notebook

The “Monolith First” Playbook.

Monolith

It is incredibly difficult to know the correct boundaries before understanding the domain.

Modular Monolith

Enforce logical boundaries without paying the cost of network distribution.

Modular Monolith

Look for business capabilities (Ordering, Catalog), not technical nouns (User, Database).

Independent Service

Extract only when the boundary is stable and autonomy is required.

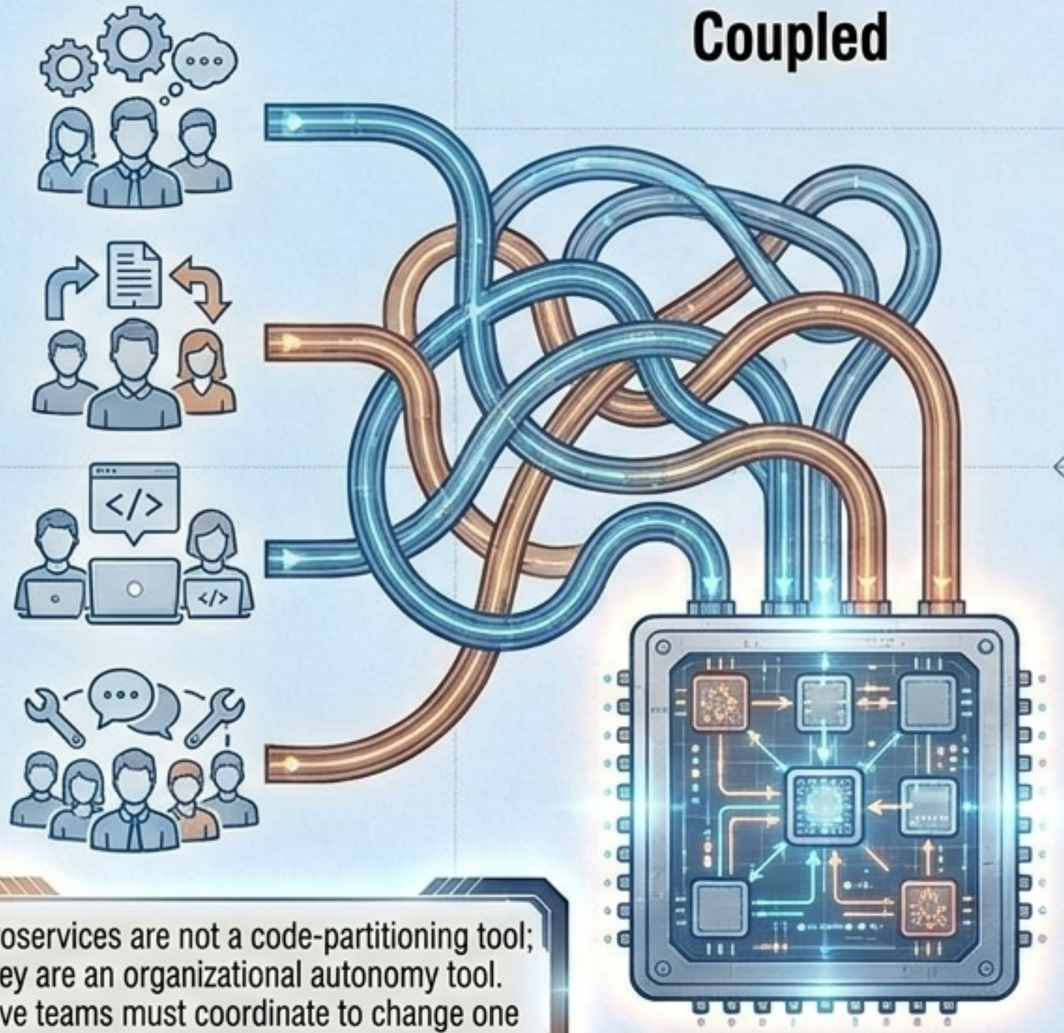
Gemini Notebook

Architecture as Organizational Structure.

Conway's Law

Organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations.

Coupled



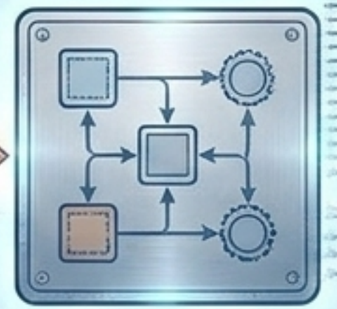
Giant Service

Microservices are not a code-partitioning tool; they are an organizational autonomy tool. If five teams must coordinate to change one service, you have organizational coupling.

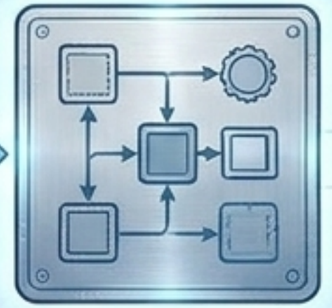
Autonomous

Team A

Team B



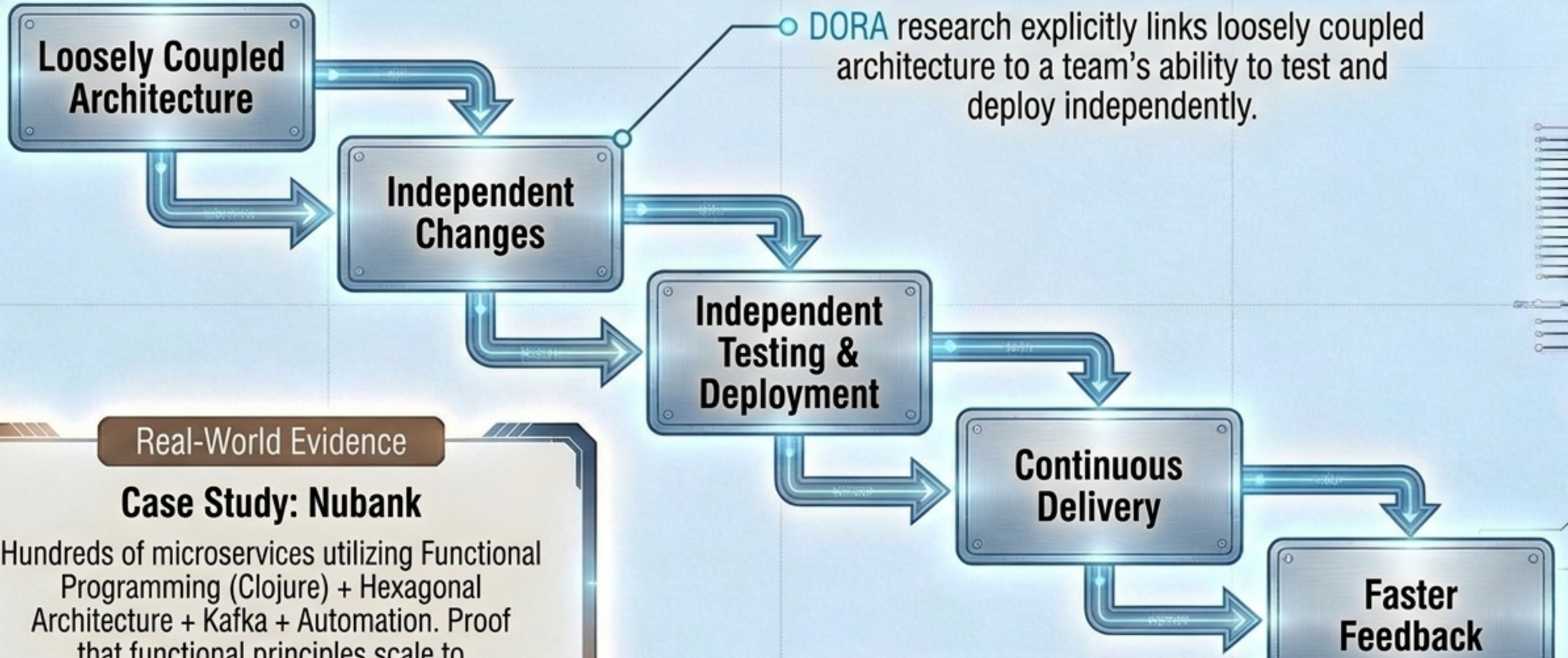
Service A



Service B

Gemini Notebook

Architecture Drives Delivery (The DORA Connection).



DORA research explicitly links loosely coupled architecture to a team's ability to test and deploy independently.

Real-World Evidence

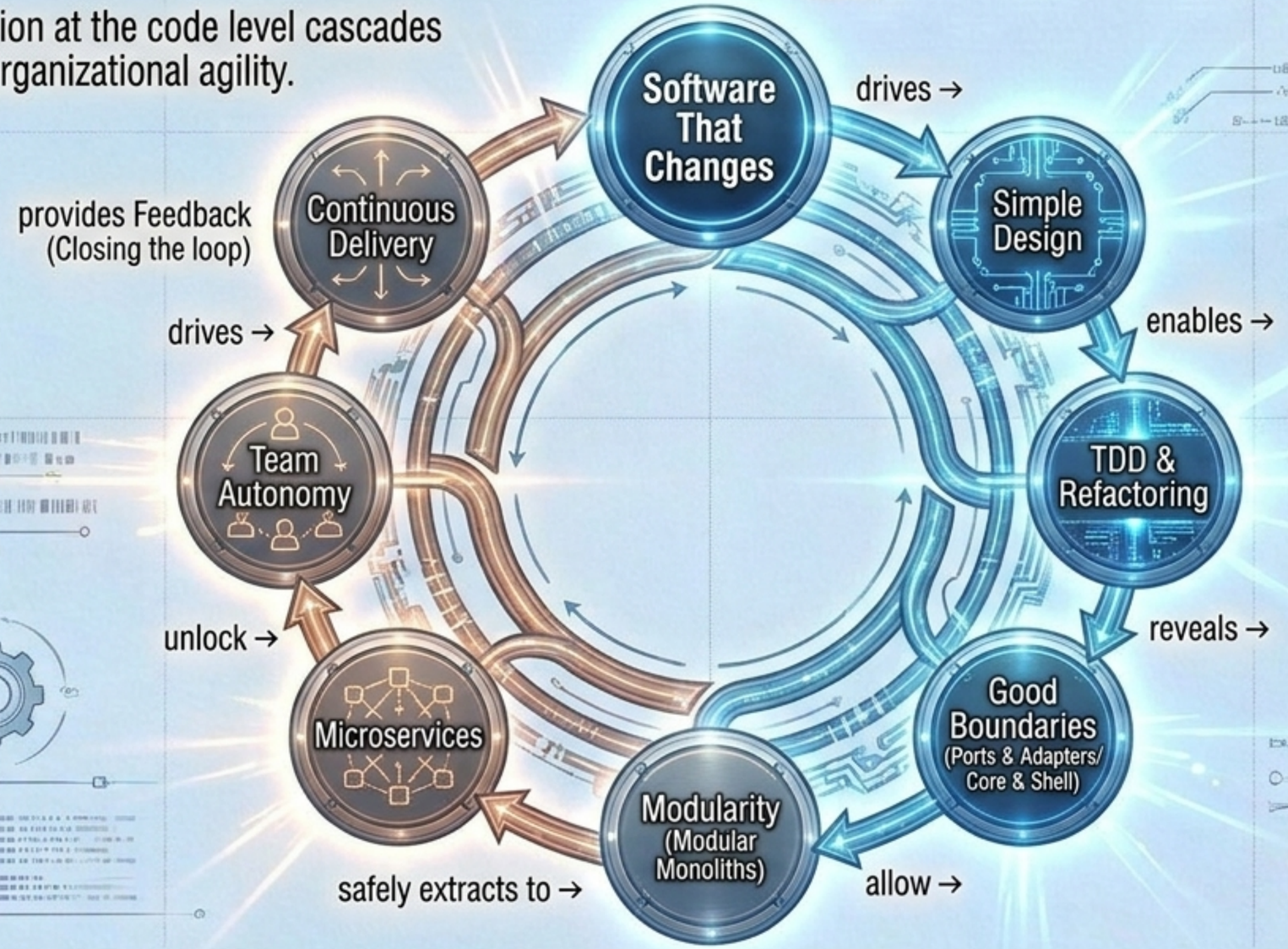
Case Study: Nubank

Hundreds of microservices utilizing Functional Programming (Clojure) + Hexagonal Architecture + Kafka + Automation. Proof that functional principles scale to enterprise distributed systems.

Gemini Notebook

The System Dynamics of Software Architecture.

Every micro-decision at the code level cascades into macro-level organizational agility.



Gemini Notebook

The Final Rule.

“Good architecture makes the important things easy to change and the unimportant things easy to ignore.”

Simple design creates local optionality. Good architecture creates systemic optionality. Do not build structures today that make it expensive to change your mind tomorrow.

Gemini Notebook